

PART III — Practical / Laboratory Examination

Total: 40 points

General Instructions

You are part of an IT infrastructure and development team tasked with improving a company's web application platform. Complete the tasks below using appropriate tools, configurations, documentation, and security practices.

You may use:

- Linux Server / Virtual Machines
- Docker or Kubernetes (optional)
- NGINX / HAProxy
- Python
- MySQL/PostgreSQL
- Postman
- Git/GitHub
- Any cloud platform or local virtualization software

CASE STUDY

“QuantumShop Online Retail Platform”

A company named **QuantumShop** is experiencing the following problems:

- The web application becomes slow during peak shopping hours.
- API endpoints are publicly accessible without proper authorization.
- Employees can access services beyond their job responsibilities.
- Development tasks are delayed because the team lacks a structured methodology.
- The company wants a scalable and secure infrastructure following Zero Trust principles.

You are hired as a junior systems and backend engineer to design and implement solutions.

Task 1: Architecture Design Diagram

(20 points)

Objective: Design the architecture following the Zero Trust Principles with Load Balancing configuration

Requirements

1. Create two backend servers:
 - Server A
 - Server B
2. Must Include in the diagram:
 - NGINX or HAProxy for Load Balancer
 - Multi-Factor Authentication
 - Least Privilege Access
 - Microsegmentation
 - Continuous Monitoring
 - Identity Verification
 - Secure API Gateway
3. Additional Requirements
 - Explain how your design prevents:
 1. Unauthorized access
 2. Lateral movement
 3. Credential misuse
4. Deliverables
 - Architecture Diagram
 - 1–2-page explanation

Rubic for Architecture Design Diagram

Criteria	Excellent	Good	Fair	Poor
Documentation (Diagram)	10	8	6	4
Technical Explanation	10	8	6	4

Task 2: Implement API Security using OAuth 2.0

(10 points)

Objective: Secure the QuantumShop API

Scenario: The QuantumShop company developed a Flask-based API for managing customer orders. However, the API currently has no authentication and authorization, making sensitive endpoints publicly accessible.

You are tasked with securing the API using OAuth 2.0 Bearer Tokens.

Existing Vulnerable Code (Python):

```
from flask import Flask, jsonify

app = Flask(__name__)

orders = [
    {"id": 1, "item": "Laptop", "customer": "Juan"},
    {"id": 2, "item": "Phone", "customer": "Maria"}
]

# PUBLIC ENDPOINT (Currently Vulnerable)
@app.route('/orders', methods=['GET'])
def get_orders():
    return jsonify(orders)

# DELETE ENDPOINT (Currently Vulnerable)
@app.route('/orders/<int:id>', methods=['DELETE'])
def delete_order(id):
    return jsonify({"message": "Order deleted successfully"})

if __name__ == '__main__':
    app.run(debug=True)
```

Tasks:

- A. Implement OAuth 2.0 Bearer Token Authentication
 - o Request without a valid token must return:

```
{
  "message": "Unauthorized"
}
```

- o Tokens must be included in the header

Authorization: Bearer <token>

B. Secure the Endpoints

Endpoint	Method
/orders	GET
/orders/	DELETE

C. Deliverables

- a. Updated Flask source code
- b. Screenshots of Successful API requests
- c. Screenshots of Failed Unauthorized requests

Rubic

Criteria	Points
Correct Token Validation	4
Proper Endpoint Protection	3
Error Handling	2
Code Quality and Readability	1

Task 3: Performance Optimization

(10 points)

Objective: Optimize API

Scenario: The QuantumShop web application is experiencing slow response times because product data is repeatedly fetched from the database every time users access the API.

As a backend developer, you are tasked with improving API performance using optimization techniques in Python.

Existing Slow API Code:

```

from flask import Flask, jsonify
import time

app = Flask(__name__)

# Simulated database query
def get_products_from_database():
    time.sleep(5) # Simulate slow database response

    return [
        {"id": 1, "name": "Laptop", "price": 50000},
        {"id": 2, "name": "Phone", "price": 25000},
        {"id": 3, "name": "Headset", "price": 3000}
    ]

@app.route('/products', methods=['GET'])
def get_products():
    products = get_products_from_database()
    return jsonify(products)

if __name__ == '__main__':
    app.run(debug=True)

```

Problem Statement:

The /products endpoint takes too long to respond because the application queries the database every time a request is made.

Optimize the API performance while maintaining correct functionality.

Tasks:

a. Apply at least ONE optimization technique to improve response time:

- Caching
- Database indexing simulation
- Query Optimization
- Compression

b. Reduce API Response Time. The optimized API should:

- Respond faster after the first request
- Avoid repeated expensive operations

- Maintain correct JSON output

c. Performance Testing. Must compare:

Test	Expected
First Request	Slower
Second Request	Faster due to optimization

d. Deliverables:

- Optimized Python source code
- Screenshot of API response times before optimization
- Screenshot of API response times after optimization
- Short explanation of:
 - Optimization technique used
 - How the optimization improved performance
 - Possible limitations of the solution

Rubic

Criteria	Points
Correct Optimization Implementation	4
Improved Response Time	3
Code Quality and Readability	2
Technical Explanation	1